Лабораторная работа №3

Шифрование гаммированием

11 Марта 2026 - Сунь Маосин

2026

Российский университет дружбы народов, Москва, Россия

Цель работы

Основная цель

В данной лабораторной работе я реализовал алгоритм шифрования гаммированием с конечной гаммой. Основная цель — изучить принципы модульной арифметики и реализовать шифрование путём сложения индексов букв текста и гаммы по модулю 33.

Реализация алгоритма

Подготовка данных

Первым шагом я создал словарь соответствия букв русского алфавита и чисел от 1 до 33. Это основа для всех дальнейших вычислений. Каждой букве присвоен порядковый номер согласно заданию.

Код словаря

```
# Russian alphabet mapping (standard order)
LETTER_TO_NUM = {
    "а": 1,  "б": 2,  "в": 3,  "г": 4,  "д": 5,  "е": 6,  "ё": 7,  "ж": 8,  "з": 9,
    "и": 10, "й": 11, "к": 12, "л": 13, "м": 14, "н": 15, "о": 16, "п": 17,
    "р": 18, "с": 19, "т": 20, "у": 21, "ф": 22, "х": 23, "ц": 24, "ч": 25,
    "ш": 26, "щ": 27, "ъ": 28, "ы": 29, "ь": 30, "э": 31, "ю": 32, "я": 33
}
```

Рис. 1: Словарь соответствия букв и чисел

Функции преобразования

Для удобства работы с текстом я создал две вспомогательные функции: `text_to_numbers` преобразует текст в список чисел, `numbers_to_text` выполняет обратное преобразование. Это позволяет легко переходить между буквенным и числовым представлением.

Код функций преобразования

```
def text_to_numbers(text):
    """Convert text to list of numbers using the alphabet mapping"""
    text = text.lower()
    numbers = []
    for char in text:
        if char in LETTER_TO_NUM:
            numbers.append(LETTER_TO_NUM[char])
        else:
            # Skip characters not in alphabet (spaces, punctuation)
            continue
    return numbers
```

Функция шифрования

Функция `gamma_encrypt` выполняет основную работу. Она преобразует текст и гамму в числа, затем для каждой буквы текста находит соответствующую букву гаммы (с циклическим повторением) и складывает их. Если сумма превышает 33, применяется операция взятия остатка, при этом остаток 0 заменяется на 33.

Код функции шифрования

```python
def gamma_encrypt(text, gamma):
    """
    Encrypt text using gamma cipher
    text: plaintext (Russian)
    gamma: key phrase (Russian)
    """
    # Convert text and gamma to numbers
    text_numbers = text_to_numbers(text)
    gamma_numbers = text_to_numbers(gamma)

    print(f"Text numbers: {text_numbers}")
    print(f"Gamma numbers: {gamma_numbers}")

    # Encrypt: (text_num + gamma_num) mod 33
    # Note: mod 33 with numbers 1-33 requires special handling
    result_numbers = []
    gamma_len = len(gamma_numbers)

    for i, text_num in enumerate(text_numbers):
        gamma_num = gamma_numbers[i % gamma_len]  # Repeat gamma if needed
        sum_num = text_num + gamma_num
```

Функция дешифрования

Функция `gamma_decrypt` выполняет обратную операцию. Из чисел зашифрованного текста вычитаются числа гаммы. Если результат меньше 1, к нему прибавляется 33 для восстановления исходного значения. Благодаря свойствам модульной арифметики, мы получаем исходный текст.

Код функции дешифрования

```
def gamma_decrypt(cipher, gamma):
    """
    Decrypt text using gamma cipher
    cipher: ciphertext (Russian)
    gamma: key phrase (Russian)
    """
    # Convert cipher and gamma to numbers
    cipher_numbers = text_to_numbers(cipher)
    gamma_numbers = text_to_numbers(gamma)

    print(f"Cipher numbers: {cipher_numbers}")
    print(f"Gamma numbers: {gamma_numbers}")

    # Decrypt: (cipher_num - gamma_num) mod 33
    result_numbers = []
    gamma_len = len(gamma_numbers)

    for i, cipher_num in enumerate(cipher_numbers):
        gamma_num = gamma_numbers[i % gamma_len]  # Repeat gamma if needed
```

Тестирование на примере

Для проверки корректности работы я использовал пример из задания: текст «ПРИКАЗ» и гамма «ГАММА». Программа выводит все промежуточные числа, что позволяет проследить процесс шифрования и убедиться в правильности алгоритма.

Код тестирования

```python
def test_example_from_assignment():
    """Test the example from the assignment"""
    print("=" * 60)
    print("TESTING EXAMPLE FROM ASSIGNMENT")
    print("=" * 60)

    # Example from the assignment
    text = "П Р И К А З"
    gamma = "Г А М М А"

    print(f"Original text: {text}")
    print(f"Gamma: {gamma}")
    print()

    # Encrypt
    encrypted = gamma_encrypt(text, gamma)
    print(f"\nEncrypted: {encrypted}")

    # Decrypt
    decrypted = gamma_decrypt(encrypted, gamma)
    print(f"\nDecrypted: {decrypted}")
```

Вывод

Вывод

В ходе лабораторной работы я успешно реализовал алгоритм шифрования гаммированием. Программа корректно выполняет шифрование и дешифрование текста на русском языке. Эксперимент подтвердил, что при наличии правильной гаммы возможно полное восстановление исходного сообщения.